

Tianjin University
Tianjin International Engineering Institute

“Dual Carbon” Themed Project-Based Course Project Report

双碳主题企业实战项目课报告

Project Name	基于本地大模型与 RAG 技术的网站智能客服系统
Major	待填写
Grade	待填写
Name	待填写
Student No.	待填写
Date	2026 年 6 月 10 日

1 Kick-off

1.1 Basic Information

Item	Content
Project Name	基于本地大模型与 RAG 技术的网站智能客服系统
Company Name	NXP AIoT Cloud / Cloud Lab 相关公开资料与课程项目场景
Team Name	本地 RAG 智能客服项目组（待填写）
Company Supervisor	待填写
Responsibility in the team	负责需求分析、知识库构建、RAG 检索链路设计、FastAPI 后端、前端演示页面、测试验收与答辩材料整理。

1.2 Project Background

随着“双碳”目标推进，工业企业、能源设备和智能制造场景对低成本、低时延、可解释的数字化工具需求不断提升。边缘 AI、端侧大模型和本地知识库问答系统能够减少云端依赖，降低数据传输与隐私风险，并帮助工程人员快速查询设备、平台和方案资料。NXP AIoT Cloud / Cloud Lab 提供了面向边缘智能、AI Hub、Model Zoo、LLM/VLM Edge Studio、Qwen/VSS、YOLOv8n、i.MX/MCU 应用范例等技术资料，是构建课程级 RAG 智能客服知识库的合适对象。

传统网站客服常见问题是回答范围固定、更新成本高、无法解释答案来源；直接调用通用大模型又容易产生幻觉，并且无法保证答案一定来自企业资料。本项目选择“本地大模型 + RAG 检索增强生成”的技术路线，把企业公开资料整理成本地半结构化知识库，用户提问时先检索依据，再由本地模型生成回答，并在网页上展示检索来源和处理流程。

项目社会价值体现在帮助用户更高效地理解边缘 AI 与智能制造资料，减少重复咨询和人工解释成本；经济价值体现在降低客服和培训成本，提高企业资料复用效率；技术价值体现在验证本地大模型、向量数据库、混合检索、可解释问答和前端流程可视化的集成方案。

1.3 Project Input

企业与项目输入主要包括课程任务书、NXP AIoT Cloud 官网公开页面、Cloud Lab 站点地图、AI Hub / Model Zoo / LLM Edge Studio / VLM Edge Studio / Qwen / VSS / YOLOv8n 等技术文档，以及登录后获取的应用范例和系统方案目录。资料被整理为 Markdown 半结构化文档，包含标题、分类、来源、采集日期、关键词和摘要，便于后续文本切分与检索。

本项目没有进行传统意义上的线下实地考察，而是通过线上资料采集、网页交互测试和本机部署实验获取输入信息。实践中发现的主要问题包括：官网资料分散、部分动态接口需要登录态、直接把长 HTML 入库会降低可读性、短中文问题容易造成向量召回偏差、课堂环境中 Ollama 或 ChromaDB 可能临时不可用。因此系统设计了本地知识库清洗、混合检索、降级机制和健康检查接口。

Input Type	Source	Use in Project
文本资料	NXP AIoT Cloud / Cloud Lab 官网与文档	构建本地 RAG 知识库
课程任务	codex 开发任务书	确定功能、接口、目录与验收标准
接口数据	应用范例与解决方案目录	补充 i.MX、MCU、系统方案知识条目
运行数据	health/stats/chat API 测试结果	验证知识库规模、模型状态和回答质量

1.4 Project Objectives and Expected Outcomes

1.4.1 Project Objectives

- 设计并实现一个可运行的网站智能客服系统，支持网页端提问、后端接收、知识库检索、本地模型生成和来源展示。
- 基于 NXP AIoT Cloud、Edge AI、LLM/VLM Edge Studio 等资料构建本地知识库，并生成向量索引。
- 默认使用本机 Ollama，模型为 qwen2.5:7b，embedding 模型为 nomic-embed-text，不依赖 OpenAI 或其他云端大模型 API。
- 在前端展示 RAG 六步流程、Top-K 检索来源、相似度、知识库统计和模型健康状态，满足 15 分钟课堂演示需要。
- 提供 ChromaDB、SimpleVectorStore、TF-IDF、MOCK_LLM 等降级方案，提高演示稳定性。

1.4.2 Expected Outcomes

Outcome	Description
软件原型	FastAPI + 原生前端 + Ollama + RAG 的可运行系统。
知识库与索引	17 篇本地知识库文档，134 个可检索 chunks，支持 ChromaDB 持久化。
文档材料	README、API Reference、Project Design、Demo Script、Technical Defense Guide。
测试结果	pytest、smoke_test、acceptance_check 均通过；关键问答场景经过真实接口验证。
答辩材料	完整课程报告与 15 分钟答辩 PPT。

1.5 Project Plan

1.5.1 Project Phase

Phase	Key Work	Deliverable
需求分析	解读任务书，确定本地 RAG 客服、接口、前端和验收要求。	需求清单、项目目录规划
资料采集	整理 NXP AIoT Cloud 官网和 Cloud Lab 相关资料。	17 篇 Markdown 知识库文档
系统设计	设计四层架构、RAG 流程、数据结构和降级机制。	project_design.md、架构图
原型实现	开发 FastAPI、RAG 模块、Ollama 客户端和前端页面。	可运行代码与网页 Demo
测试优化	验证接口、检索质量、异常降级和页面适配。	测试脚本、修复记录
总结答辩	整理报告、PPT、演示脚本和答辩问答。	报告与答辩材料

1.5.2 Schedule Plan

Time	Task	Output
第 1 阶段	需求分析与技术选型	明确 Python + FastAPI + Ollama + RAG + 原生前端方案
第 2 阶段	知识库采集与清洗	构建 17 篇本地 Markdown 文档
第 3 阶段	后端与 RAG 实现	完成 document_loader、chunker、embedding、vector_store、retriever、prompt_builder
第 4 阶段	前端页面与交互实现	三栏演示页面、示例问题、流程状态和来源展开
第 5 阶段	模型部署与索引验证	Ollama qwen2.5:7b、nomic-embed-text、Chroma 索引
第 6 阶段	测试、优化与答辩准备	通过脚本测试，形成报告与 PPT

1.5.3 WBS

WBS ID	Work Package	Owner	Output
1.0	需求与资料分析	待填写	需求说明、资料来源清

			单
2.0	知识库构建	待填写	Markdown 知识库、元数据
3.0	RAG 检索模块	待填写	chunk、embedding、向量库、混合召回
4.0	后端 API	待填写	health/stats/rebuild/chat 接口
5.0	前端展示	待填写	网页 Demo、流程可视化
6.0	测试与验收	待填写	pytest、smoke、acceptance 结果
7.0	报告与答辩	待填写	项目报告、PPT、演示脚本

2 Data Study

2.1 Requirement Analysis

2.1.1 Function Requirements

- 网页端支持用户输入问题、选择示例问题、设置 Top-K 并发送。
- 后端支持健康检查、知识库统计、示例问题、重建索引、RAG 聊天五类核心接口。
- 系统能够从本地知识库中检索相关资料，并返回标题、类别、来源、相似度和内容片段。
- 系统能够构造受约束 Prompt，调用本地 Ollama 模型生成回答，并列岀参考资料。
- 系统能够识别资料数量、知识库范围、系统能力等运行上下文问题，避免无关检索。
- 系统对无依据问题应明确说明，不编造来源。

2.1.2 Performance Requirements

性能要求主要包括可用性、稳定性、响应速度和检索准确性。课堂演示场景中，系统应在普通实验室电脑上可运行；本地模型生成时间可接受；知识库重建应返回 JSON 结果，不出现前端解析失败；检索结果应优先命中与问题主题匹配的文档，例如 RAG 问题优先命中“RAG 智能客服问答流程”。

2.1.3 Appearance and HCI Requirements

前端采用三栏布局：左侧状态面板用于展示 Ollama、模型、知识库和向量库状态；中间为聊天区和输入区；右侧展示 RAG 六步流程和检索来源。页面需要适合投屏演示，文字不能超出容器，示例问题和底部输入窗口要自动换行，来源内容可展开查看。

2.1.4 Safety and Environmental Requirements

本项目为软件系统，无传统机械安全风险。安全要求主要体现在数据安全、模型调用边界和运行稳定性：默认不调用云端大模型 API，资料保存在本地；对未入库的实时天气、股票、新闻等问题明确说明没有依据；Ollama 不可用时返回友好提示，避免后端崩溃。环保角度上，本地 RAG 系统减少重复人工咨询和不必要网络调用，符合数字化节能和高效知识管理方向。

2.1.5 Cost and Budget Requirements

项目成本控制目标是尽量使用开源工具和本地已有硬件。主要软件依赖包括 FastAPI、ChromaDB、numpy、python-dotenv、Ollama 等，均可免费使用。硬件成本取决于本地运行 qwen2.5:7b 的电脑配置；如果机器性能不足，可启用 MOCK_LLM 或更小模型完成教学演示。

2.2 Literature Review

2.2.1 Design Theory Fundamental Research

本项目采用用户中心设计和工程原型迭代方法。用户中心设计强调从答辩观众、教师和资料查询者的视角出发，让系统不仅能回答问题，还能展示“为什么这样回答”。工程原型迭代方法强调先实现最小可运行闭环，再逐步优化知识库、检索效果、异常处理和页面体验。

2.2.2 Technical research

RAG (Retrieval-Augmented Generation) 通过外部知识检索增强大模型回答能力。与直接生成相比，RAG 可以引入最新或私有资料，减少幻觉，并让答案具备来源可追溯性。Ollama 使本地部署大模型更加便捷；ChromaDB 提供向量持久化和相似度检索；FastAPI 适合构建结构化接口；原生 HTML/CSS/JS 能降低课堂部署复杂度。

2.2.3 Market Research

工业软件、企业知识库、智能客服和边缘 AI 平台都在向“可解释、本地化、低依赖”的方向发展。企业内部资料通常存在分散、更新快、人工查找成本高的问题，RAG 智能客服可以把分散文档变为可问答的知识服务。对于 NXP AIoT Cloud 等技术平台，用户需要快速理解不同工具、模型、开发板和系统方案之间的关系，因此本地化知识问答具有实际应用价值。

2.2.4 Best Practice and Lessons Learned

成功实践表明，RAG 系统不能只追求模型回答流畅，还要关注资料清洗、chunk 粒度、检索质量、Prompt 约束和前端可解释性。失败案例通常来自三类问题：资料质量差导致检索噪声大；没有降级机制导致演示环境稍有问即不可用；没有来源展示导致回答难以验证。本项目针对这些风险设计了半结构化 Markdown、混合召回、降级机制和来源可视化。

3 Design Proposal

3.1 Technical Approach and Methodology

技术路线为“本地资料入库 → 文本切分 → embedding 向量化 → ChromaDB 持久化 → 混合检索 → Prompt 构造 → Ollama 本地生成 → 前端展示来源与流程”。该路线兼顾可运行性、可解释性和演示稳定性。

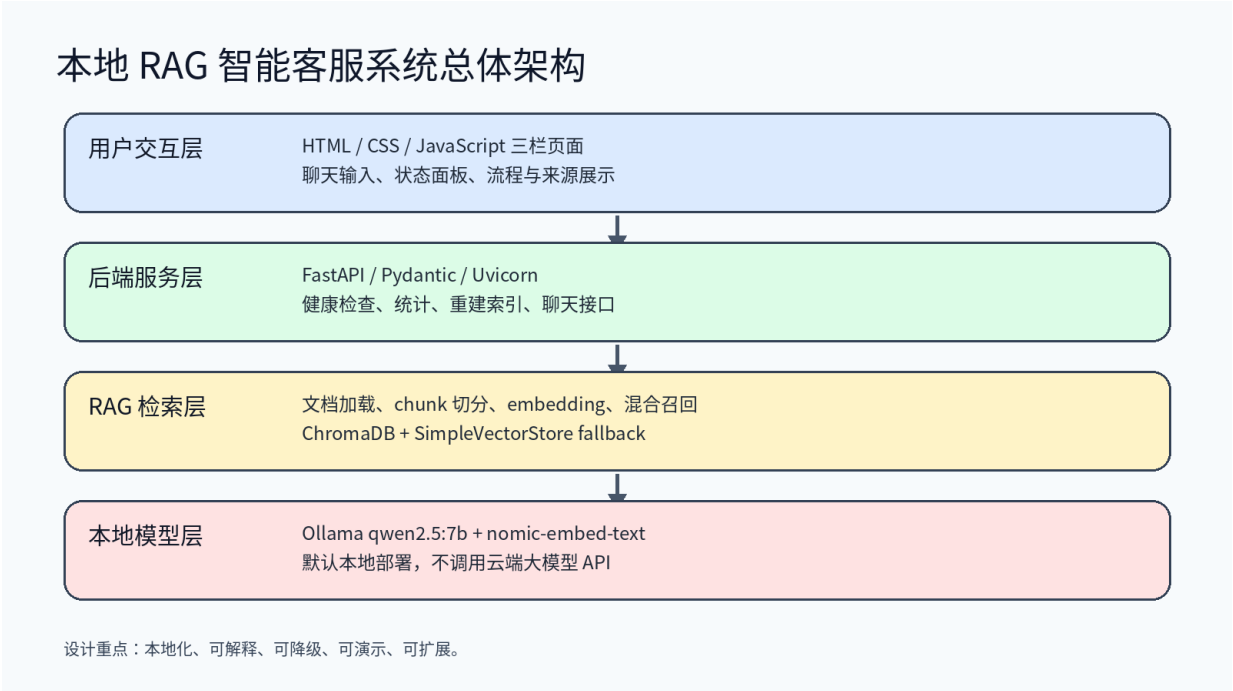


图 3-1 系统总体架构图

3.2 Design Methodology

设计方法分为需求拆解、模块化设计、原型实现、测试反馈和迭代优化。系统按职责拆成配置、Ollama 客户端、文档加载、切分、embedding、向量库、检索器、Prompt Builder、接口和前端页面，避免单文件堆叠，便于答辩说明和后续维护。

3.3 Manufacturing Process

本项目为软件原型，制造过程对应软件构建与部署过程。首先搭建 Python 虚拟环境并安装依赖；其次编写后端与 RAG 模块；然后整理知识库并构建向量索引；最后启动 FastAPI 服务并通过浏览器访问演示页面。与传统制造相比，软件制造的关键控制参数是依赖版本、模型可用性、索引状态、接口响应和页面交互。

3.4 Testing Methods

测试采用单元测试、脚本验收、真实接口测试和人工演示检查相结合的方法。pytest 覆盖 chunk、prompt、schema、向量检索、运行上下文识别和关键词召回；smoke_test 覆盖基础 API；acceptance_check 覆盖任务书验收项；真实接口测试覆盖资料数量、RAG 流程、NXP 技术问题、越界问题和索引重建。

4 Concept Design

4.1 Idea Creation

- 方案 A：普通 FAQ 静态客服。优点是实现简单，缺点是覆盖范围有限、无法处理自然语言变体。
- 方案 B：直接调用通用大模型。优点是回答流畅，缺点是容易幻觉、无法保证依据来自项目资料。
- 方案 C：云端大模型 + RAG。效果较好，但依赖网络和 API，不符合本地演示与数据不出本机目标。
- 方案 D：本地 Ollama + RAG + 前端可视化。兼顾本地化、可解释和课堂演示，因此作为最终方案。

4.2 Concept Voting

Concept	Accuracy	Local Deployment	Explainability	Demo Stability	Result
FAQ 静态客服	中	高	中	高	不选，智能程度不足
直接大模型	中	中	低	中	不选，来源不可追溯
云端大模型 + RAG	高	低	高	中	不选，依赖外部 API
本地 Ollama + RAG	高	高	高	高	最终方案

4.3 Functional and Performance Design

功能设计包括五个核心闭环：知识库管理、索引构建、问答生成、来源展示和异常降级。性能设计重点不是追求最高吞吐量，而是在课程电脑上稳定运行；系统使用 Top-K=3 控制 Prompt 长度，使用 MIN_SCORE 过滤低相关来源，使用 hybrid reranking 改善短中文问题和专有名词召回。

4.4 Concept Draft

概念草图可以抽象为三栏页面：左侧是系统状态和索引操作，中间是聊天窗口和输入框，右侧是 RAG 流程与检索来源。用户视线从左到右可以依次看到“系统是否可用、用户问了什么、系统如何处理、答案依据来自哪里”。

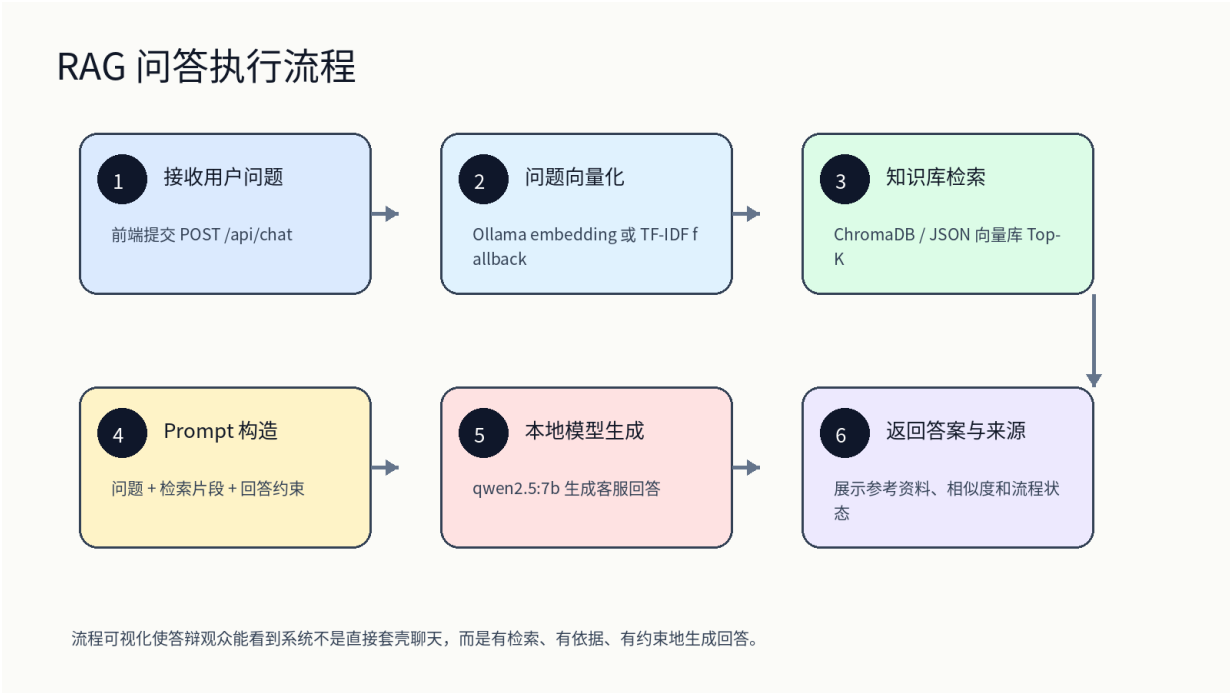


图 4-1 RAG 问答流程概念图

5 Detailed Design

5.1 Appearance Design

页面视觉风格采用科技感但克制的深浅搭配，适合课堂投屏。布局强调信息密度和可读性：状态卡片简洁，聊天区重点突出问答内容，流程区用步骤状态展示系统内部链路，来源卡片包含标题、分类、相似度和内容片段。前端修复了示例问题和底部输入区域不换行的问题，保证长中文文本不会超出屏幕。

5.2 Structure Design

Layer	Module	Main Files
前端交互层	页面、样式、接口调用	frontend/index.html, style.css, app.js
后端服务层	API、流程状态、异常处理	app/main.py, schemas.py
模型调用层	Ollama chat/embed/health	app/ollama_client.py
RAG 层	加载、切分、embedding、检索、prompt	app/rag/*.py
数据层	知识库与向量库	data/knowledge_base, data/vector_store

5.3 Function Design

核心功能实现方式如下：后端启动时读取配置，API 接收请求后检查索引状态；普通业务问题进入 Retriever，Retriever 同时使用向量检索和关键词标题匹配进行混合召回；系统类问题则直接使用后端运行上下文回答；Prompt Builder 根据问题类型选择是否注入运行上下文；OllamaClient 调用本地 qwen2.5:7b 生成回答；前端根据响应展示答案、来源、流程和指标。

5.4 Digital Design Model Creation

数字设计模型主要由代码模块、知识库文档、向量索引和接口响应模型组成。Pydantic schema 定义 ChatRequest、ChatResponse、SourceItem、ProcessStep、HealthResponse 等结构，保证前后端数据清晰；Markdown 知识库作为可维护的内容模型；ChromaDB 持久化向量索引作为检索模型。

6 Prototype Manufacturing

6.1 Manufacturing Methods

软件原型采用本地开发与迭代调试方式完成。使用 Python 虚拟环境隔离依赖，FastAPI 负责服务，Ollama 负责模型运行，ChromaDB 保存向量。前端无需 Node.js 构建，直接由 FastAPI 挂载静态文件，降低部署和演示复杂度。

6.2 Material Preparation

Material / Tool	Purpose
Python 3.12 虚拟环境	运行后端、测试脚本和依赖
FastAPI / Uvicorn	提供 Web 服务和 API
Ollama qwen2.5:7b	本地生成客服回答
nomic-embed-text	生成问题和文档向量
ChromaDB	持久化向量检索
Markdown 知识库	保存 NXP AIoT Cloud 等资料
HTML/CSS/JS	实现课堂演示页面

6.3 Manufacturing Process

制作过程包括：创建项目目录；编写配置文件和 schemas；实现 Ollama 客户端；实现文档加载、chunk 切分、embedding provider、vector store 和 retriever；实现 Prompt Builder；编写 FastAPI 接口；实现前端三栏页面；整理知识库；构建索引；运行测试；根据测试反馈优化检索、提示词和页面换行。

6.4 Assembly and Commissioning

组装调试阶段重点检查各模块连接关系：前端能否访问后端；后端能否读取知识库；embedding 模型能否生成向量；向量库是否能重建；chat 接口是否返回 answer、sources、process 和 metrics；Ollama 不可用时是否友好降级。调试后系统已支持 17 篇知识库文档和 134 个 chunks。

7 Prototype Testing

7.1 Testing Methods

测试环境为本地 Linux 工作站，项目路径为 nxp-aiot-local-chatbot。测试方法包括命令行脚本测试、API 真实调用、前端人工检查和异常场景模拟。



图 7-1 测试与验收闭环

7.2 Function Testing

Test Case	Expected Result	Status
GET /api/health	返回模型、知识库、向量库状态	通过
GET /api/stats	返回 17 篇文档、134 个 chunks 和分类	通过
POST /api/rebuild-index	返回 success，并重建 Chroma/Ollama 索引	通过
POST /api/chat: RAG 智能客服	首源命中 RAG 智能客服问答流程	通过

POST /api/chat: 资料数	使用系统运行上下文回答，不返回无关 sources	通过
POST /api/chat: 天气/股票	明确无依据，sources 为空	通过

7.3 Performance Testing

性能测试关注响应时延和稳定性。真实接口测试中，资料数量类问题无需向量化，通常能快速返回；RAG 技术问题需要 embedding、检索和模型生成，耗时主要取决于 qwen2.5:7b 本地推理速度。索引重建接口可在本地完成 17 篇文档、134 个 chunks 的重建，返回 JSON 结果，避免前端 JSON.parse 错误。

7.4 User Experience Testing

用户体验测试覆盖页面打开、示例问题点击、输入框长文本换行、来源卡片展示、RAG 流程状态变化和异常提示。经过修复后，底部输入窗口和示例问题区域能够正确换行，长问题不会超出屏幕；来源展示按文档聚合，避免同一标题重复干扰阅读。

8 Prototype Optimization

8.1 Problem Shooting

Problem	Analysis	Solution
短问法召回不稳定	纯向量检索对“介绍一下 rag 智能客服”等短中文问题易偏移	加入关键词、标题覆盖率和混合 reranking
资料数问题回答死板或来源错误	系统状态问题不应进入普通 RAG 检索	增加运行上下文问题识别与专用 Prompt 模式
重建索引 JSON.parse 错误	后端异常时前端收到非预期响应	修复 rebuild-index 响应和 Chroma readonly 清理逻辑
来源标题重复	同一文档多个 chunk 分开返回	按 doc_id 聚合来源，每个文档最多合并两个片段
前端文本超屏	示例问题和输入区域缺少换行约束	修复 CSS 换行和布局约束

8.2 Design Optimization

优化后的系统将问题分为两类：系统运行上下文问题和领域资料问题。前者直接依据后端实时统计回答，例如资料数量、知识库范围和系统能力；后者进入 RAG 检索。检索端采用向量召回与关键词标题匹配融合，Prompt 端强调只列实际使用的参考资料。这样既提升了自然问法适应性，也降低了无关来源污染。

8.3 Iteration Optimization

后续迭代方向包括：增加网页上传知识库和后台管理；支持增量索引而非全量重建；增加多轮会话和历史摘要；加入重排序模型提升复杂问题召回；对接真实边缘设备或开发板演示；增加回答质量评分与用户反馈闭环。

9 Defense Presentation

9.1 Summary Report

本项目完成了基于本地大模型与 RAG 技术的网站智能客服系统。系统围绕 NXP AIoT Cloud / Cloud Lab 和边缘 AI 资料构建本地知识库，使用 Ollama 本地模型生成回答，使用 ChromaDB 保存向量索引，并通过前端页面展示回答、来源、相似度、流程和模型状态。项目验证了本地化 RAG 在企业资料问答、课堂演示和边缘 AI 场景中的可行性。

Final Item	Completion
代码项目	完整 FastAPI + RAG + Ollama + 原生前端项目
知识库	17 篇资料、134 个 chunks、覆盖 NXP AIoT Cloud 等主题
模型与索引	qwen2.5:7b、nomic-embed-text、ChromaDB、fallback 机制
测试	pytest 8 passed, smoke_test 和 acceptance_check 通过
文档	README、API、项目设计、演示脚本、技术答辩指南

9.2 Presentation Preparation

答辩 PPT 按 15 分钟汇报设计，建议结构为：项目背景与需求 2 分钟，系统目标与技术路线 2 分钟，架构与 RAG 流程 4 分钟，知识库与实现细节 3 分钟，测试结果与演示 2 分钟，总结与展望 2 分钟。演示时重点展示网页左侧状态、右侧六步流程、Top-K 来源、资料数量问答、RAG 流程问答和越界问题处理。

答辩时可强调三个核心亮点：第一，本项目不是普通大模型套壳，而是可解释的本地 RAG 系统；第二，系统有多级降级机制，适合课堂环境；第三，知识库已扩展到 NXP AIoT Cloud 官方资料和应用范例目录，具备继续扩展为企业资料客服的基础。

注：封面中的 Major、Grade、Name、Student No.、Team Name 和 Company Supervisor 等个人或团队信息需根据实际提交要求补充。其余项目内容、技术实现、测试与答辩准备已按当前项目状态完成。